AutoCodeBench: Benchmarking Repository-Level Code Completion in Automotive Software Systems

Anonymous Authors[1]

Abstract

Large Language Models (LLMs) have achieved remarkable success on general coding benchmarks, yet their performance in safety-critical vertical domains remains underexplored. We introduce *AutoCodeBench*, the first benchmark for repository-level code completion in Software-Defined Vehicles (SDV). Unlike general-purpose software, automotive code challenges LLMs through proprietary APIs, strict hardware-bound constraints, and safety protocols absent from public training corpora. Derived from 58 production-grade real-world repositories, AutoCodeBench comprises 4,044 multi-language tasks (C/Java) that strictly require cross-file context resolution. Our experiments reveal a significant Domain Knowledge Gap, with state-of-the-art models suffering up to 40% accuracy drops compared to general benchmarks. Notably, we observe a Leaderboard Inversion: models excelling in general coding (e.g., Qwen2.5Coder) underperform legacy models (CodeLlama) in this vertical domain due to distribution shifts. We further demonstrate that our domain-adapted structure-aware retrieval (Graph-RAG) effectively achieves average 22.4% relative improvement in EM over direct generation by employing signature-centric abstraction and deterministic topological traversal.[1]

1. Introduction

Large Language Models (LLMs) have revolutionized software engineering, with models like GPT-5 (OpenAI), Gemini (Google), and CodeLlama (Rozière et al., 2024) achieving remarkable proficiency on general coding benchmarks (Chen, 2021; Austin et al., 2021). However, deploying these models in safety-critical vertical domains, where incorrect code can cause physical harm, remains a critical open challenge. Unlike web applications where bugs lead to degraded user experience, errors in automotive software can result in vehicle malfunctions, safety violations, or even loss of life (Kocher et al., 2020).

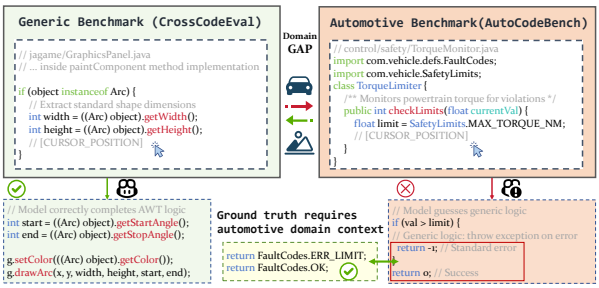


Figure 1. The domain gap. Left: The model succeeds on generic UI logic. Right: In a safety-critical powertrain task, it reverts to generic conventions (returning `-1`), failing to utilize the imported `FaultCodes` or enforce `SafetyLimits`. This illustrates how general pre-training overlooks the proprietary safety protocols essential for automotive software.

Automotive software, within the paradigm of Software-Defined Vehicles (SDV) (Pretschner et al., 2007), exhibits three domain-specific characteristics that distinguish it from general-purpose code. First, automotive code relies on *Proprietary API Ecosystems*: specialized interfaces such as `CarPropertyManager` for vehicle property access and `VehicleHAL` for hardware abstraction are absent from public training corpora like The Stack (Kocetkov et al., 2022), forming the rare knowledge that LLMs do not memorize. Second, automotive logic suffers from *Hardware-Bound Structural Coupling*: unlike self-contained algorithms, a single function call in a cockpit application may depend on struct definitions in distant header files, Android Interface Definition Language (AIDL) interface declarations, and hardware register mappings. Third, automotive development is governed by *Safety-Critical Constraints*: adherence to ISO 26262 (Debouk, 2019) functional safety standards means that API misuse, such as invoking `setFloat()` with incorrect `VehiclePropertyIds`, does not merely cause runtime errors but can violate safety invariants.

Despite these unique challenges, existing benchmarks fail to evaluate LLMs in this vertical domain. Function-level benchmarks like HumanEval (Chen, 2021) and MBPP (Austin et al., 2021) test isolated algorithmic reasoning, ignoring cross-file dependencies entirely. While repository-level benchmarks such as RepoBench (Liu et al.,

[1]All the experimental data and code are available at: `https://anonymous.4open.science/r/AutoCodeEval/`.

For ICML 2026 reviewers: When reviewing this paper, please follow the policy for the use of LLMs as specified on your Reviewer Console. Note that the assigned LLM policy might differ from the policy you indicated during reviewer registration.

2023) and CrossCodeEval (Ding et al., 2023) incorporate project context, they are constructed from general-purpose repositories (e.g., web frameworks, data tools) where APIs are well-documented and internalized during pre-training. This leaves a critical question unanswered: *How do state-of-the-art LLMs perform when strictly constrained by the proprietary, hardware-bound logic of a vertical industry?*

To bridge this gap, we introduce *AutoCodeBench*, the first benchmark dedicated to domain-specific code completion in the automotive industry. Curated from 58 high-quality open-source automotive repositories, AutoCodeBench comprises 4,044 rigorously constructed test cases spanning two dominant languages: C (vehicle middleware and Hardware Abstraction Layer) and Java (Android Automotive OS cockpit applications). Each test case is designed via static analysis to strictly require cross-file context, such as HAL struct definitions or framework base classes, ensuring that success demands genuine understanding of automotive software topology rather than pattern matching.

Our experiments across 8 widely used LLMs reveal two findings. (1) *Domain Knowledge Gap:* LLMs suffer up to 40% lower Exact Match accuracy on automotive code compared to general benchmarks, with failures concentrated on proprietary APIs absent from training data, as models frequently hallucinate plausible-looking but non-existent methods. (2) *Leaderboard Inversion:* surprisingly, models excelling in general coding benchmarks (e.g., Qwen2.5Coder (Hui et al., 2024), DeepSeekCoder (Guo et al., 2024)) underperform legacy models like CodeLlama in this vertical domain, revealing a systematic blind spot in current LLM development. Furthermore, we demonstrate that Graph-RAG, a structure-aware retrieval approach that traverses project dependency graphs, substantially outperforms semantic vector search, confirming that bridging the domain gap requires structural reasoning rather than mere text similarity.

In summary, this work makes three contributions:

- **A New Benchmark.** We release AutoCodeBench, comprising 4,044 domain-specific test cases from 58 automotive repositories. Each task targets proprietary API usage in C middleware (HAL, CAN, VHAL) and Java cockpit applications (AAOS framework), with verified cross-file dependencies.
- **Comprehensive Empirical Analysis.** We quantify the Domain Knowledge Gap: LLMs exhibit up to 40% lower accuracy on automotive code, and we identify Leaderboard Inversion where general-purpose optimization fails to transfer to vertical domains.
- **A Reference Approach.** We propose Graph-RAG, a structure-aware retrieval approach that improves performance by up to average 22.4% relative improvement in EM over direct generation through precise dependency-graph traversal, establishing a baseline for future research.

2. Benchmark Construction

As shown in Figure 2, we first curate automotive repositories from GitHub, then apply language-specific static analysis to identify cross-file dependencies, and finally enforce quality control to ensure benchmark validity.

2.1. Phase 1: Domain-Specific Data Curation

To ensure the benchmark captures the vertical depth of the automotive industry, we curated data from permissively licensed open-source repositories, organizing them into two subsets that mirror the industry's stratified architecture.

Vehicle Middleware Subset (C Language). We targeted the safety-critical layer responsible for hardware abstraction and signal processing. To distinguish from general algorithmic C code, we utilized domain-specific keywords such as *VHAL*, *ECU*, *CAN bus*, and *SOME/IP* to filter repositories. Following a rigorous manual review by domain experts, we retained 22 high-quality repositories. These projects encapsulate complex vehicle hardware abstractions and macro-heavy protocol logic, representing domain-specific patterns essential for vehicle control that are fundamentally distinct from standard algorithmic C code.

Intelligent Cockpit Subset (Java Language). We focused on the AAOS ecosystem, filtering for projects explicitly interacting with the `android.car` framework. This yielded 36 repositories covering domains such as CarService, HVAC, and IVI. Unlike standard mobile apps, these projects exhibit tight coupling with vehicle properties (e.g., `VehiclePropertyIds`), ensuring the benchmark evaluates understanding of the vehicle interface rather than generic Android logic.

2.2. Phase 2: Context-Aware Task Construction

Unlike generic coding tasks where logic is often self-contained, automotive domain knowledge is inherently non-local. API contracts and signal formats are physically segregated from implementation logic in external headers or base classes. To capture these dependencies, we devised two static analysis pipelines.

Java: Tree-sitter Analysis and Task Formulation. Cockpit logic relies heavily on inheritance and framework callbacks. Simple regex matching fails to capture these polymorphic dependencies. We use tree-sitter-java (tree-sitter) to build a project-wide Fully Qualified Class Name (FQCN) index. We explicitly isolate non-local dependencies via a "Dependency Stripping + Compilation Difference" strategy: internal dependency classes are replaced with dummy shells (retaining only type names) to trigger compilation errors on specific external symbol usages. Using the compiler's error caret coordinates, we slice the target code to ensure

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

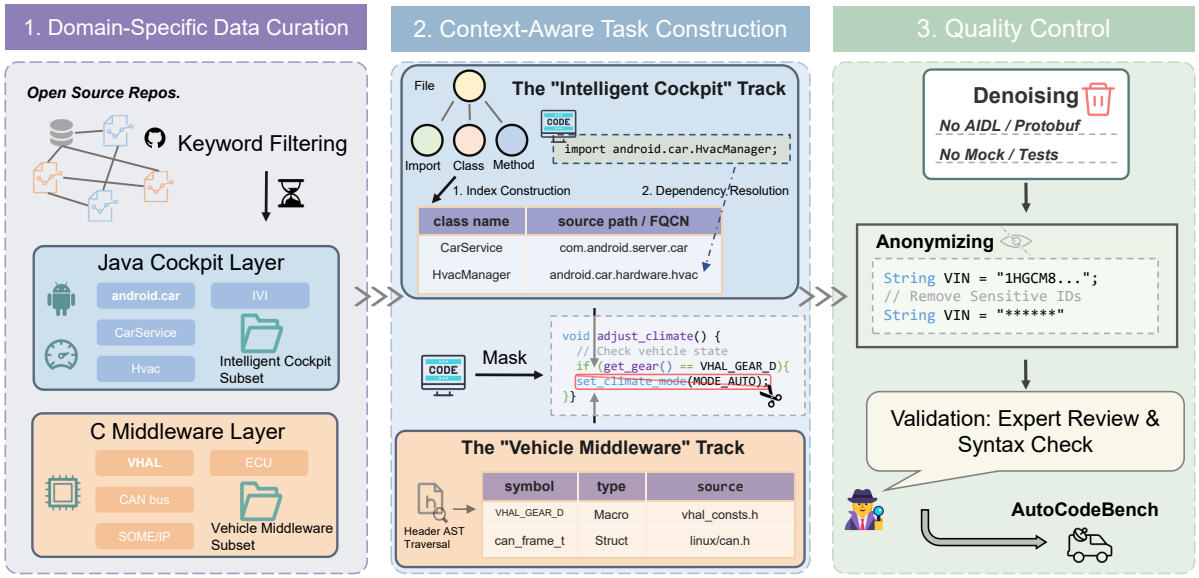


Figure 2. The overall construction pipeline of AutoCodeBench.

the completion task strictly necessitates retrieving valid API contracts from the cross-file context.

C: Libclang-based Symbol Resolution and Task Formulation. The automotive C domain presents challenges due to hardware-specific macros and non-unified build systems. We first use libclang (Lattner & Adve, 2003) to parse all header files and construct a global Symbol Table of functions, structs, and macros. To strictly enforce context dependence, we employ a Header Masking strategy: in a shadow build environment, we selectively comment out domain-critical `#include` directives. We then compile the source files and analyze the resulting "undefined symbol" errors. These error locations confirm where external HAL definitions (e.g., VHAL structs) are strictly required. Similar to the Java pipeline, we use these verified failure points to slice the task, ensuring the model must resolve the missing hardware context to generate syntactically valid code.

2.3. Phase 3: Safety-Critical Quality Control

Given the safety-critical nature of the automotive domain, data quality necessitates strict standards for privacy and relevance. We enforced a three-tier quality control protocol. First, through Domain-Specific Filtering, we removed non-human-authored artifacts (e.g., auto-generated AIDL, Protobuf) and excluded mock/test directories to ensure the benchmark reflects genuine, production-grade engineering intent. Second, we applied Privacy Desensitization via a rigorous scanning protocol to detect and sanitize sensitive automotive identifiers, including VINs, proprietary CAN signal databases (DBC), and diagnostic IP addresses. Finally, we conducted Manual Expert Validation. Beyond automated syntax checks, we prioritized data quality through rigorous manual reviews by domain experts, verifying that tasks require reasoning about complex automotive states and API contracts, strictly filtering out trivial predictions to guarantee high-quality evaluation standards.

3. Dataset Characteristics and Analysis

3.1. Benchmark Statistics and Test Density

AutoCodeBench is constructed from 58 real-world automotive repositories, yielding a total of 4,044 validated test cases. Table 1 presents the detailed statistics of the benchmark.

A notable characteristic of our dataset is the High Test Density. We define Avg. Cases / Files as the average number of valid completion tasks extracted per source file. As shown in Table 1, the Java subset exhibits a density of 4.40 cases per file, while the C subset shows 3.41. This high density indicates that automotive implementation files are structurally rich, containing multiple distinct points of interaction with external APIs (e.g., HAL interfaces or framework callbacks) within a single compilation unit. Unlike benchmarks that may extract only one function per file, our approach captures the multifaceted logic typical of production-grade modules.

3.2. Knowledge Distribution Shift

To quantify the domain gap, we analyze the API distribution within the Intelligent Cockpit subset, addressing the frequency bias inherent in general-purpose LLMs. Our analysis reveals a distinct Inversion of Frequency in AutoCodeBench: APIs that are inherently niche due to their exclusivity to the automotive platform which naturally scarce in general mobile-centric corpora become the dominant Head entities here. As illustrated in Figure 3 and Figure 4, the dataset is overwhelmingly dominated by the `android.car.*`

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 1. Statistics of the AutoCodeBench benchmark.

Dataset Subset	#Repo.	#File	#Test	Avg. Context	Max. Context	Avg. Cases / Files
AutoCodeBench (C)	22	977	3,336	1,531.22	77,902	**3.41**
AutoCodeBench (Java)	36	161	708	712.24	4,708	**4.40**

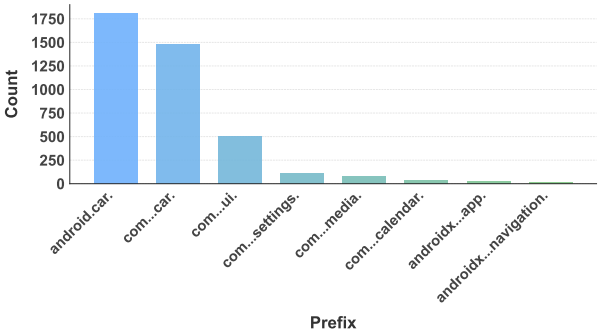


Figure 3. Distribution of API Prefixes in the Java Subset.

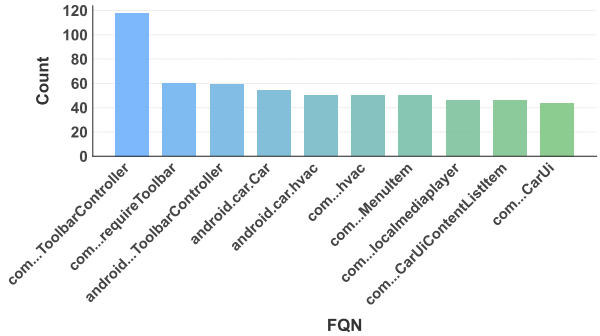


Figure 4. Top-10 API FQN Occurrences.

namespace (unavailable in the standard Android SDK) and specialized primitives like `CarPropertyManager`, while generic Android APIs appear with significantly lower frequency.

This distribution shift explains the root cause of API Hallucination. When generic LLMs encounter automotive contexts, they probabilistically revert to high-frequency generic patterns because the correct automotive ontology resides in their "forgotten" long-tail knowledge. For example, they may hallucinate a `setTemp()` method instead of the required `setFloat(VehiclePropertyIds.HVAC_TEMPERATURE_SET, ...)`. Furthermore, the significant presence of internal `com.android.car.*` components compels models to reason about proprietary system logic absent from public documentation, confirming that AutoCodeBench constitutes a rigorous test of a model's ability to overcome frequency bias via precise context retrieval.

3.3. Domain-Specific Functional Topology

To demonstrate the representational integrity of AutoCodeBench, we analyze the hierarchical functional topology visualized in the sunburst charts (Figure 5a for C and Figure 5b for Java). The Intelligent Cockpit (Java) subset focuses on the AAOS framework. The distribution balances user-facing IVI modules, such as Radio and Media, with critical System Services including `CarService` and `PowerManagement`. This composition ensures the benchmark evaluates the orchestration of vehicle properties rather than simple UI logic.

Complementing this, the Vehicle Middleware (C) subset encapsulates the safety-critical middleware architecture. The topology features a high density of VHAL implementations and real-time Communication Protocol Stacks such as CAN and SOME/IP. These components interface high-level commands with low-level ECU operations. This hierarchical diversity confirms that AutoCodeBench spans the vertical stack of automotive software, preventing overfitting to generic syntax and testing domain intelligence within automotive architectural constraints.

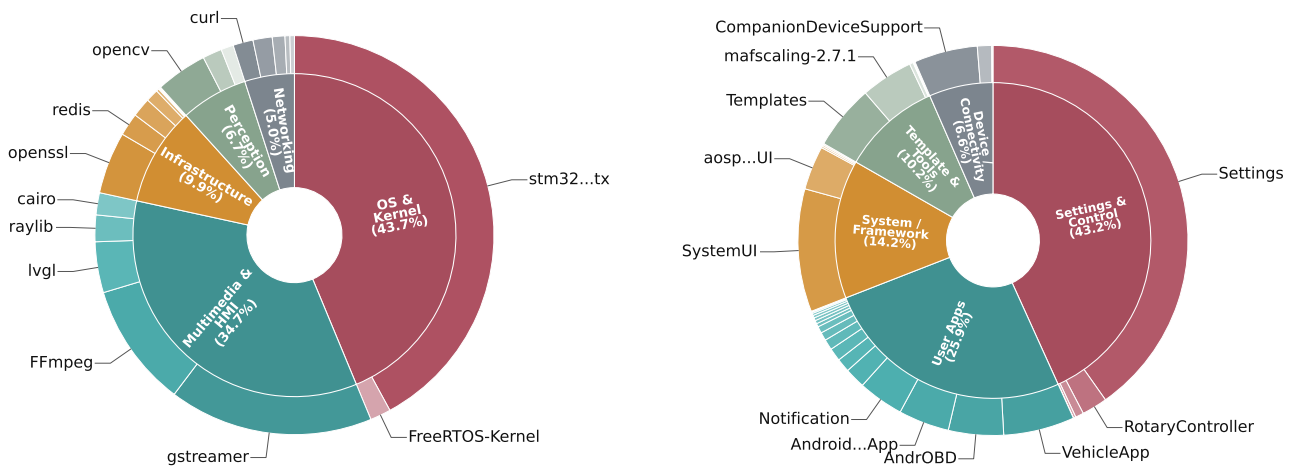
4. Graph-RAG: A Reference Approach

Conventional RAG relies on semantic similarity, which often retrieves structurally irrelevant functions in automotive code. To establish a strong baseline demonstrating the value of structural context, we propose Graph-RAG, a retrieval framework designed to navigate the heterogeneous dependency topology of intelligent cockpits.

Structure-Aware Graph Construction. we employ a polyglot parsing strategy leveraging robust static analysis tools. Specifically, for the C Middleware layer, we use libclang (Lattner & Adve, 2003) to parse the Abstract Syntax Tree (AST) and resolve macro-heavy header dependencies. For the Java Application Layer, we employ tree-sitter (tree-sitter) to resolve Fully Qualified Class Names (FQCN) and explicitly model inheritance chains. Crucially, we resolve implicit dependencies (e.g., Android wildcard imports) into explicit edges via deterministic heuristics, converting the project into a traversable graph topology.

Deterministic Retrieval & Abstraction. Unlike probabilistic vector search, Graph-RAG performs a Deterministic Depth-Limited Traversal starting from the focal file. It follows `#include` paths or class hierarchies to fetch precise defining contexts required for compilation. To maximize token efficiency and adherence to safety protocols, we employ Signature-Centric Abstraction: instead of raw implementations, we extract only API contracts (e.g., function prototypes, struct layouts). This strictly constrains the LLM's generation space to valid, safety-critical interfaces,

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.



(a) Distribution in C subsets

(b) Distribution in Java subsets

Figure 5. Distribution of test cases across source repositories in different languages.

minimizing hallucination risks.

5. Experimental Setup

Models. We evaluate 8 widely used models, ranging from leading open-source code specialists (including the CodeLlama-7B (Rozière et al., 2024), DeepSeekCoder-6.7B (Guo et al., 2024), StarCoder2-7B (Lozhkov et al., 2024), and Qwen2.5Coder-7B (Hui et al., 2024)) to general-purpose engines (including the GPT-5.1 (OpenAI), Qwen3-8B (Yang et al., 2025), Llama3-8B (Dubey et al., 2024) and Gemini-3-pro (Google)) to ensure a comprehensive assessment across different architectures and training strategies.

Evaluation Metrics. We adopt two primary metrics to measure the performance in a safety-critical domain.

- **Exact Match (EM):** Measures the percentage of generations that are syntactically identical to the reference solution (ignoring whitespace). In automotive software, where API contracts are rigid, high EM is a strong proxy for functional correctness.
- **Edit Similarity (ES):** Calculated based on the Levenshtein distance, ES captures the semantic closeness of the generated code to the ground truth, recognizing partial success where the model retrieves the correct logic but misses minor syntactic details.

Retrieval Baselines. We design three comparative settings to isolate the contribution of structural knowledge.

- **Direct Generation:** The model relies solely on its parametric memory given the function signature and docstring, serving as a baseline to measure the "Domain Knowledge Gap".
- **Baseline RAG (Vector Baseline):** We adapt the standard RAG pipeline (Lewis et al., 2020) by implementing LangChain's (Langchain) retrieval toolkit specifically for our automotive corpus. Code snippets are embedded into dense vectors (Karpukhin et al., 2020), and relevant context is retrieved via cosine similarity search, representing the current industry standard for generic retrieval.
- **Graph-RAG (Ours):** Our proposed method that utilizes static analysis to traverse the project's AST and `Include/Import` graph, deterministically retrieving the precise definitions of dependencies.

6. Results and Analysis

We evaluate our AutoCodeBench on eight widely used LLMs. Our analysis aims to answer three research questions: (1) *How do general-purpose and code-specialized LLMs perform on domain-specific automotive code?* (2) *Does retrieving cross-file context improve performance, and which retrieval strategy is more effective?* (3) *What are the common failure modes of LLMs in this vertical domain?* We begin with a broad overview of the main results, followed by a detailed analysis of retrieval efficacy and error types.

6.1. RQ1: Main Results on Code Completion

Table 2 presents the comparative performance of representative models. The results reveal a harsh reality: current LLMs struggle significantly in the automotive domain, exposing a critical "Vertical Domain Gap".

Overall Trends and Benchmark Value. The overall performance across all models is startlingly low compared to general coding benchmarks. In the Direct Generation setting, even the powerful proprietary model Gemini-3-pro only achieves an Exact Match (EM) score of 53.57% on the C subset, while GPT-5.1 follows with 38.97%. Similarly, in the Java subset, the highest direct generation scores are 53.39% (Gemini-3-pro) and 45.62% (StarCoder2-7B). This stands

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 2. Main results: code completion performance on AutoCodeBench. Values are reported in percentage (%). Best results in each category are **bolded**.

		Direct Generation		Baseline RAG		Graph-RAG (Ours)	
Model	**Lang.**	**EM (%)**	**ES (%)**	**EM (%)**	**ES (%)**	**EM (%)**	**ES (%)**
CodeLlama-7B	C	32.43	59.07	41.12	69.34	**42.26**	**69.35**
	Java	44.92	68.93	**47.88**	**72.38**	47.05	72.35
DeepSeekCoder-6.7B	C	17.51	49.36	29.23	62.04	**31.90**	**63.13**
	Java	43.64	68.65	**58.24**	**81.55**	47.19	73.65
Qwen2.5Coder-7B	C	25.00	58.77	28.63	61.31	**30.72**	**61.95**
	Java	36.02	62.38	26.04	44.23	**40.79**	**68.96**
StarCoder2-7B	C	22.57	50.38	**33.97**	**64.95**	32.94	63.64
	Java	45.62	69.67	47.27	**71.29**	**47.32**	71.24
Qwen3-8B	C	15.53	50.19	**33.97**	**64.95**	23.56	58.70
	Java	34.04	62.67	34.18	64.42	**34.57**	**64.60**
Llama3-8B	C	14.84	48.53	23.40	54.55	**24.01**	**56.04**
	Java	35.45	64.47	**36.78**	**65.47**	36.30	64.55
GPT-5.1	C	38.97	66.17	44.45	**70.62**	**45.71**	69.68
	Java	44.07	70.20	**45.20**	70.30	45.06	**70.86**
Gemini-3-pro	C	53.57	75.74	**58.78**	**78.50**	57.25	77.53
	Java	53.39	75.30	**55.93**	**76.75**	54.94	76.35

in stark contrast to benchmarks like HumanEval (Chen, 2021) or MBPP (Austin et al., 2021), where similar models routinely achieve pass rates exceeding 70%-80%. This dramatic performance drop quantitatively validates that AutoCodeBench is not merely testing generic algorithmic logic, but rather domain-specific compliance. The low baselines indicate that general pre-training data severely lacks the knowledge of proprietary automotive architectures, making this benchmark a rigorous testbed for vertical intelligence.

Leaderboard Inversion: General SOTA *vs.* Vertical Reality. Analyzing the performance distribution reveals intriguing anomalies that contradict general leaderboard trends. Models like Qwen2.5Coder and DeepSeekCoder are widely recognized as state-of-the-art open-source models, typically outperforming CodeLlama-7B by significant margins on general benchmarks like HumanEval and MBPP. However, on AutoCodeBench, this ranking is starkly inverted. In the C subset, CodeLlama-7B achieves 32.43% EM, significantly surpassing Qwen2.5Coder-7B (25.00%) and DeepSeekCoder-6.7B (17.51%). Crucially, this trend persists in the Java subset, where CodeLlama-7B (44.92%) continues to outperform DeepSeekCoder-6.7B (43.64%) and notably leads Qwen2.5Coder-7B (36.02%). This consistent inversion suggests a systematic *Distribution Shift*: the aggressive optimization for general coding proficiency does not linearly transfer to specialized vertical domains.

Language Divergence (C *vs.* Java). A granular look at the data highlights a structural disparity between the two sub-domains. Open-source code models perform consistently better on the Java subset compared to the C subset. For instance, DeepSeekCoder-6.7B shows a massive disparity, scoring 43.64% on Java but plummeting to 17.51% on C. Similarly, StarCoder2-7B drops from 45.62% (Java) to 22.57% (C). We attribute this to the nature of the ecosystems: the Java code is based on Android Automotive, which shares design patterns with the vast Android open-source ecosystem seen during training. In contrast, the C subset involves interaction with HAL and proprietary `struct` definitions that act as opaque interfaces to physical hardware. The models' inability to infer these hardware constraints without explicit context results in a much higher failure rate in C, further underscoring the necessity of our benchmark for testing embedded system reasoning.

6.2. RQ2: Efficacy and Efficiency of Retrieval Strategies

Impact of Contextual Retrieval. The introduction of retrieval mechanisms consistently improves performance across all metrics, validating the hypothesis that domain-specific context is essential for vertical tasks. As shown in Table 2, both Baseline RAG and Graph-RAG significantly outperform Direct Generation. For instance, in the C subset, DeepSeekCoder-6.7B sees its Exact Match (EM) score jump from 17.51% (Direct) to 29.23% with Baseline RAG, and further to 31.90% with Graph-RAG. Similarly, CodeLlama-7B improves from 32.43% to 41.12% and 42.26%, respec-

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 3. Build and retrieval time of RAG strategies.

Method	Build Time	Retrieval Time
Baseline RAG	49,072s	5s
Graph-RAG	**1,537s**	**1s**

tively. This trend holds true for Edit Similarity (ES) as well, where Graph-RAG consistently boosts the ES of DeepSeek-Coder on C from 49.36% to 63.13%. These substantial gains confirm that bridging the "Vertical Domain Gap" requires injecting proprietary API definitions and structural dependencies that can mitigate the hallucination issue in LLMs.

Superiority of Graph-RAG: Precision and Efficiency. Comparing the two RAG strategies reveals the distinct superiority of Graph-RAG in both stability and efficiency. We quantify the benefits by calculating the Average Relative Improvement (ARI) over Direct Generation. In the challenging C subset, Graph-RAG achieves an ARI of 39.9% across all models. While Baseline RAG shows a higher average (44.9%) due to outlier gains on specific models (e.g., Qwen3), Graph-RAG outperforms it on 5 out of 8 models. In the Java subset, Graph-RAG yields a steady 4.9% improvement, surpassing Baseline RAG (3.5%), which suffers from negative transfer on models like Qwen2.5Coder (-27.7%). More critically, Table 3 highlights a drastic difference in computational efficiency. Graph-RAG reduces the index construction (Build) time from over 13 hours (49,072 s) to roughly 25 minutes (1,537 s), and retrieval latency from 5 s to 1 s. This efficiency stems from avoiding the expensive computation of vector embeddings for the entire repository, making Graph-RAG a viable solution for real-time coding assistants in large-scale automotive projects.

Exploratory Analysis: Performance *vs.* Task Difficulty. To investigate the limits of current models, we stratify the test cases by "difficulty," defined as the number of unique external API calls within a file. As illustrated in Figure 6, we analyze the performance of Qwen2.5Coder-7B (with Graph-RAG) on the Java subset across increasing difficulty buckets. We observe a clear monotonic degradation: as the number of external dependencies grows from the $[0, 4)$ bucket to $[12, +\infty)$, the Exact Match (EM) score drops sharply from 52.48% to 33.78%. This trend suggests that while Graph-RAG effectively aids in context retrieval, highly coupled files—which require synthesizing numerous dispersed definitions—impose a heavy reasoning burden that challenges even state-of-the-art models, highlighting the hardest frontier in automotive code completion.

6.3. RQ3: Case Study: Failure Modes Analysis

To understand *why* models fail, we conducted a rigorous manual audit of 100 randomly sampled failure cases from

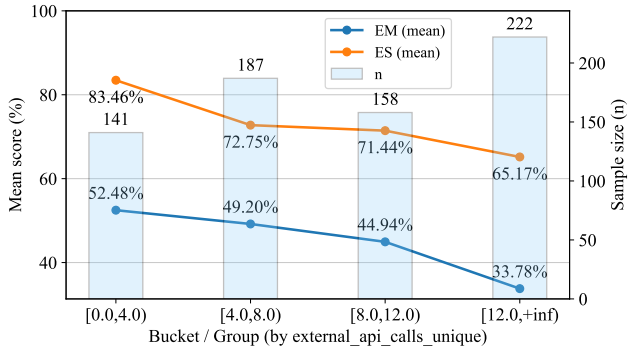


Figure 6. Performance monotonically degrades as the number of unique external API calls increases.

the Vehicle (C) and Cockpit (Java) subsets. We categorize the errors into three distinct failure modes. We manually counted 100 error samples and found API Hallucination is the dominant error type (49%), followed by Type/Struct Mismatch (43%) and Safety Protocol Violation (8%). We provide comprehensive qualitative examples and detailed case studies for each category in Appendix D.

(1) API Hallucination (Frequency Bias). As visualized in Figure 7, this is the most prevalent failure. Models tend to invent plausible-looking but non-existent methods based on generic naming conventions. For instance, in the `TorqueMonitor` task, the model reverts to a generic error handling pattern (`return -1;`), ignoring the specific `FaultCodes.ERR_LIMIT` constant defined in the project's internal dependencies. This reflects a bias towards high-frequency public programming patterns, bypassing the low-frequency, proprietary interfaces required by the project.

(2) Strict Type & Struct Mismatch. In the C middleware subset, models frequently fail to comply with strict hardware definitions. A common failure mode involves passing generic primitives (e.g., `int`) where a specific `typedef struct` or `enum` is required. *Example:* A model generates `hal_write(0x01)` (generic integer), while the VHAL signature strictly requires a pointer to a hardware-mapped struct: `hal_write(&VHAL_GEAR_P)`. This highlights the model's inability to resolve "opaque types" defined in distant header files without retrieval augmentation.

(3) Safety Protocol Violation. Automotive software adheres to strict state-machine constraints. We observe that general LLMs often generate code that is syntactically correct but functionally unsafe because it skips mandatory state checks. *Example:* In a `CarPowerManager` task, the model directly invokes `shutdown()` without wrapping it in the required `if (isVehicleStationary())` check. Such violations pose catastrophic risks in real-world deployment, underscoring the gap between coding capability and safety compliance.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

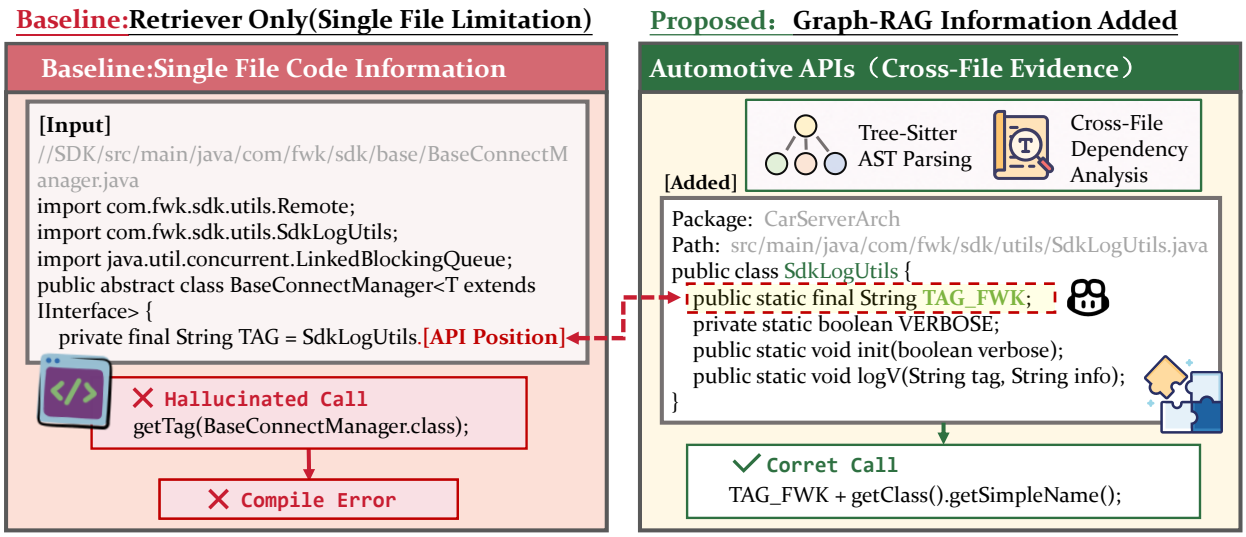


Figure 7. Case study. Graph-RAG prevents API hallucination by providing the correct method signature from the Automotive APIs.

7. Related Work

Code LLMs in Vertical Domains. While LLMs like GPT-5 (OpenAI)and DeepSeekCoder (Guo et al., 2024) excel in general software engineering, their capability in specialized vertical domains remains largely unassessed. Unlike general IT software, automotive code under the Software-Defined Vehicle (SDV) paradigm (Broy, 2006) presents unique challenges, including heterogeneous architectures (e.g., Java apps interacting with C/C++ HALs (Macario et al., 2009)) and strict safety protocols (ISO 26262) (Debouk, 2019). Existing studies typically overlook these constraints, focusing on generic algorithmic logic. Our work bridges this gap by evaluating LLMs specifically within the safety-critical ecosystem of Intelligent Cockpits.

Benchmarking Repository-Level Code Completion. The evaluation of Code LLMs has shifted from function-level tasks (Chen, 2021) to repository-level assessments. However, existing benchmarks like (Ding et al., 2023) and RepoBench (Liu et al., 2023) rely on general-purpose repositories, failing to capture the "long-tail" distribution of domain-specific APIs (e.g., `android.car.*`) and intricate cross-language dependencies (e.g., JNI, IPC) inherent in vehicle systems. AutoCodeBench addresses this limitation as the first benchmark curated from high-quality automotive projects to test "Vertical Domain Intelligence".

Structure-Aware Retrieval for Code. RAG is essential for handling the large context of software projects (Lewis et al., 2020). However, standard sparse or dense retrieval methods often lack the precision required for embedded development, where resolving deep structural dependencies (e.g., class hierarchies, hardware definitions) is critical to prevent system failures. We advance this area by demonstrating that structure-aware retrieval (Graph-RAG) is indispensable for navigating the complex dependency graphs of automotive software, surpassing generic semantic retrieval methods.

8. Limitation

Our work has several limitations that warrant discussion. First, AutoCodeBench focuses exclusively on C and Java, the two dominant languages in intelligent cockpit development, excluding emerging paradigms such as Rust for safety-critical systems or Kotlin for modern AAOS applications. Second, while we curate from production-grade open-source repositories, proprietary OEM codebases may exhibit different patterns, particularly regarding internal APIs and stricter compliance requirements. Third, we employ static metrics (EM, ES) rather than execution-based validation, as automotive middleware lacks standardized test harnesses. This may overestimate functional correctness for syntactically valid but semantically incorrect completions.

9. Conclusion

We present AutoCodeBench, the first repository-level benchmark tailored for the automotive domain. Our evaluation exposes a critical domain knowledge gap and a leaderboard inversion, demonstrating that general-purpose coding proficiency does not transfer linearly to hardware-bound, safety-critical tasks. To address this, we validate that Graph-RAG which is a structure-aware retrieval strategy, significantly outperforms semantic search, achieving a 22.4% relative improvement in accuracy with 5× lower latency. These findings underscore the necessity of structural context for vertical code intelligence, establishing AutoCodeBench as a rigorous benchmark.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Impact Statement

This work introduces a benchmark and a structure-aware retrieval approach for repository-level code completion in automotive software. The benchmark itself is descriptive and does not deploy models in safety-critical systems. Potential positive impacts include enabling more rigorous evaluation of LLMs in high-stakes domains, highlighting failure modes such as API hallucination, and motivating safer tooling for developers. Potential risks include dual-use misuse of code-generation systems in safety-critical contexts or overreliance on automated outputs without adequate verification. We mitigate these risks by emphasizing evaluation rather than deployment, reporting failure modes explicitly, and recommending human-in-the-loop review for any real-world use. Overall, we expect the primary impact to be improved scientific understanding of model limitations in safety-critical software.

References

Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Al-Jaroodi, J. and Mohamed, N. Service-oriented middleware: A survey. *Journal of network and computer applications*, 35(1):211–220, 2012.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Broy, M. Challenges in automotive software engineering. In *Proceedings of the 28th international conference on Software engineering*, pp. 33–42, 2006.

Bunzel, S. Autosar–the standardized software architecture. In *International Conference on Engineering of Complex Computer Systems*, 2011.

Charette, R. N. This car runs on code. *IEEE spectrum*, 46 (3):3, 2009.

Chen, M. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Debouk, R. Overview of the second edition of iso 26262: Functional safety—road vehicles. *Journal of System Safety*, 55(1):13–21, 2019.

Ding, Y., Wang, Z., Ahmad, W., Ding, H., Tan, M., Jain, N., Ramanathan, M. K., Nallapati, R., Bhatia, P., Roth, D., et al. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. *Advances in Neural Information Processing Systems*, 36:46701–46723, 2023.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv e-prints*, pp. arXiv–2407, 2024.

Google. Gemini language model. https://gemini.google.com.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y., et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

Hui, B., Yang, J., Cui, Z., et al. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38, 2023.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. R. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Karpukhin, V., Oguz, B., Min, S., Lewis, P. S., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pp. 6769–6781, 2020.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.

Kocetkov, D., Li, R., Allal, L. B., Li, J., Mou, C., Ferrandis, C. M., Jernite, Y., Mitchell, M., Hughes, S., Wolf, T., et al. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.

Kocher, P., Horn, J., Fogh, A., Genkin, D., Gruss, D., Haas, W., Hamburg, M., Lipp, M., Mangard, S., Prescher, T., et al. Spectre attacks: Exploiting speculative execution. *Communications of the ACM*, 63(7):93–101, 2020.

Lai, Y., Li, C., Wang, Y., Zhang, T., Zhong, R., Zettlemoyer, L., Yih, W.-t., Fried, D., Wang, S., and Yu, T. Ds-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning*, pp. 18319–18345. PMLR, 2023.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Langchain. Langchain. https://www.langchain.com/.

Lattner, C. and Adve, V. A compilation framework for lifelong program analysis and transformation. In *CGO*, volume 4, pp. 75, 2003.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.

Liu, T., Xu, C., and McAuley, J. Repobench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091*, 2023.

Lozhkov, A., Li, R., Allal, L. B., Cassano, F., Lamy-Poirier, J., Tazi, N., Tang, A., Pykhtar, D., Liu, J., Wei, Y., et al. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.

Macario, G., Torchiano, M., and Violante, M. An in-vehicle infotainment software architecture based on google android. In *2009 IEEE International Symposium on Industrial Embedded Systems*, pp. 257–260. IEEE, 2009.

Neelakantan, A., Xu, T., Puri, R., Radford, A., Han, J. M., Tworek, J., Yuan, Q., Tezak, N., Kim, J. W., Hallacy, C., et al. Text and code embeddings by contrastive pre-training. *arXiv preprint arXiv:2201.10005*, 2022.

Nijkamp, E., Pang, B., Hayashi, H., et al. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*, 2022.

OpenAI. Gpt-5 language model. https://chat.openai.com.

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., and Karri, R. Asleep at the keyboard? assessing the security of github copilot's code contributions. *Communications of the ACM*, 68(2):96–105, 2025.

Pretschner, A., Broy, M., Kruger, I. H., and Stauner, T. Software engineering for automotive systems: A roadmap. In *Future of Software Engineering (FOSE'07)*, pp. 55–71. IEEE, 2007.

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Ferrer, C. C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., and Synnaeve, G. Code llama: Open foundation models for code, 2024. URL https://arxiv.org/abs/2308.12950.

Shrivastava, D., Larochelle, H., and Tarlow, D. Repository-level prompt generation for large language models of code. In *International Conference on Machine Learning*, pp. 31693–31715. PMLR, 2023.

tree-sitter. tree-sitter. https://github.com/tree-sitter/tree-sitter.

Wang, J., Luo, X., Cao, L., He, H., Huang, H., Xie, J., Jatowt, A., and Cai, Y. Is your ai-generated code really safe? evaluating large language models on secure code generation with codeseceval. *arXiv preprint arXiv:2407.02395*, 2024.

Ward, D. and Street, W. Guidelines for the use of the c language in vehicle based software. 1998.

Xia, C. S., Wei, Y., and Zhang, L. Automated program repair in the era of large pre-trained language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 1482–1494. IEEE, 2023.

Yamaguchi, F., Golde, N., Arp, D., and Rieck, K. Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE symposium on security and privacy*, pp. 590–604. IEEE, 2014.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Zhang, F., Chen, B., Zhang, Y., Keung, J., Liu, J., Zan, D., Mao, Y., Lou, J.-G., and Chen, W. Repocoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570*, 2023.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

A. Comprehensive Related Works

A.1. Software Engineering for Intelligent Cockpits

The transition to Software-Defined Vehicles (SDV) has repositioned the Intelligent Cockpit as the central nervous system of modern automobiles, managing user interaction, connectivity, and vehicle control (Broy, 2006). This domain presents unique software engineering challenges characterized by heterogeneity, strict safety standards, and complex architectural layering.

Heterogeneous Architecture and Middleware. Unlike homogeneous web or mobile applications, intelligent cockpits rely on a multi-layered, mixed-criticality architecture (Pretschner et al., 2007). The application layer typically runs on high-level operating systems like Android Automotive OS (AAOS), utilizing Java or Kotlin to deliver rich user experiences. However, these applications must interact with the vehicle's underlying Electronic Control Units (ECUs) through a rigid Hardware Abstraction Layer (HAL) and VehicleHAL (VHAL) interfaces, predominantly written in C/C++ (Macario et al., 2009). This necessitates robust inter-process communication (IPC) mechanisms, such as Android Binder or SOME/IP (Scalable service-Oriented MiddlewarE over IP) (Al-Jaroodi & Mohamed, 2012), creating deep cross-language dependencies that are difficult for general-purpose LLMs to resolve without context. Additionally, the widespread adoption of the AUTOSAR (AUTomotive Open System ARchitecture) standard (Bunzel, 2011) introduces complex, standardized interfaces for separating application software from hardware, further increasing the structural complexity of the codebase.

Functional Safety and Coding Standards. Automotive software is safety-critical; failures can lead to life-threatening consequences. Development is strictly governed by functional safety standards such as ISO 26262 (Debouk, 2019), which mandates rigorous development lifecycles (ASIL classification) and traceability. To mitigate risks, the industry enforces strict coding guidelines like MISRA C (Motor Industry Software Reliability Association) (Ward & Street, 1998) to prevent undefined behaviors and memory safety issues. These constraints mean that code generated for automotive systems must not only be syntactically correct but also compliant with specific safety idioms (e.g., avoiding dynamic memory allocation in critical paths), a nuance often missed by models trained on permissive web data.

Domain-Specific Ecosystems. The automotive domain possesses a vast, specialized vocabulary and API ecosystem. Concepts such as CAN bus signals, diagnostic protocols (OBD-II, UDS), and sensor fusion algorithms are pervasive. The "Long-tail" distribution of these domain-specific APIs (Charette, 2009) creates a significant knowledge gap for general LLMs, motivating the need for a dedicated benchmark like AutoCodeBench that specifically targets these proprietary and standardized interfaces.

Evolution of Automotive Software Complexity. The automotive software landscape has evolved dramatically with the transition to SDVs. Early work by Broy (Broy, 2006) and Pretschner et al. (Pretschner et al., 2007) established the foundational challenges of automotive software complexity. Modern vehicles contain over 100 million lines of code (Charette, 2009), necessitating sophisticated development practices. Android Automotive OS (Macario et al., 2009) has emerged as a dominant platform for IVI systems, introducing new challenges in integrating mobile frameworks with vehicle control. Service-oriented architectures (Al-Jaroodi & Mohamed, 2012) and Hardware Abstraction Layers add layers of complexity. Spectre-class vulnerabilities (Kocher et al., 2020) have heightened concerns about security in connected vehicles, further motivating the need for correct-by-construction code generation.

A.2. Benchmarking Code LLMs

The evaluation of Code LLMs has evolved from function-level to project-level assessments.

- **Single-File Benchmarks:** HumanEval (Chen, 2021) and MBPP(Austin et al., 2021) serve as the standard for evaluating algorithmic reasoning. However, they operate in a "zero-context" vacuum. Models achieving high scores on these benchmarks may still exhibit misleading performance when deployed in vertical domains, as they lack the ability to integrate with the large-scale, API-heavy codebases found in automotive systems.
- **Repository-Level Benchmarks:** CrossCodeEval (Ding et al., 2023) and RepoBench (Liu et al., 2023) represent a significant leap by incorporating cross-file context. However, these datasets are derived from general-purpose repositories (e.g., web frameworks, data science tools). They suffer from a significant distribution shift compared to automotive software, which relies heavily on middleware logic (e.g., signal debouncing, state machine management) and domain-specific SDKs (e.g., `android.car.*`). AutoCodeBench addresses this by providing a strictly curated set of automotive repositories, focusing on the unique interplay between cockpit apps and vehicle services.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

A.3. Code Intelligence and Retrieval-Augmented Generation

Foundation Models for Code. The evolution of Code LLMs has progressed from early models like CodeGen (Nijkamp et al., 2022) and AlphaCode (Li et al., 2022) to more capable architectures. StarCoder (Li et al., 2023) and StarCoder2 (Lozhkov et al., 2024) leverage The Stack (Kocetkov et al., 2022) for training on permissively licensed code. CodeLlama (Rozière et al., 2024) extends Llama with Fill-in-the-Middle (FIM) capabilities. DeepSeekCoder (Guo et al., 2024) and Qwen2.5Coder (Hui et al., 2024) represent the current state-of-the-art in open-source code models. GPT-4 (Achiam et al., 2023) demonstrates strong zero-shot coding capabilities. However, these models are predominantly trained on web-centric codebases, creating blind spots for vertical domains with proprietary APIs.

Retrieval-Augmented Generation for Code. To handle the "infinite context" of large software projects, Retrieval-Augmented Generation (RAG) is standard. While sparse (BM25) and dense (vector) retrieval work for natural language, they often miss precise code relationships. In automotive embedded development, precision is paramount; retrieving an incorrect driver interface definition can lead to critical system failures. Recent works suggest that structural retrieval—leveraging Abstract Syntax Trees (AST) or Call Graphs—is essential for strongly typed languages (Ding et al., 2023). RepoCoder (Zhang et al., 2023) introduces iterative retrieval for repository-level completion, while RLPG (Shrivastava et al., 2023) leverages prompt generation strategies. Code embeddings via contrastive learning (Neelakantan et al., 2022) and Code Property Graphs (Yamaguchi et al., 2014) have shown promise for vulnerability detection and code understanding. We implement a Graph RAG baseline explicitly modeling the import and call relationships in automotive projects, showing that structural retrieval is critical for resolving the deep dependencies in cockpit architectures.

Domain-Specific Evaluation. While general code benchmarks proliferate, domain-specific evaluation remains nascent. DS-1000 (Lai et al., 2023) targets data science libraries, but operates at the function level. SWE-Bench (Jimenez et al., 2024) evaluates real-world GitHub issue resolution but focuses on general repositories. Research on LLMs in safety-critical domains (Wang et al., 2024) highlights concerns about hallucination (Ji et al., 2023) and security vulnerabilities (Pearce et al., 2025). Automated program repair (Xia et al., 2023) has shown promise but rarely addresses embedded constraints. The catastrophic forgetting phenomenon (Kirkpatrick et al., 2017) suggests that models optimized for general code may lose specialized knowledge, partially explaining the Leaderboard Inversion we observe.

B. Benchmark Construction Details

B.1. List of Source Repositories

B.1.1. Source Repositories for C

The selected source repositories are categorized into several key domains relevant to automotive software engineering. **Autonomous Driving & AI** covers core stacks and algorithms including *Autonomous Driving*, *AD Algorithms*, *AD Machine Learning*, *AD Perception*, and *AD Simulation*. **Automotive Systems & Software** includes domain-specific platforms such as *Automotive Operating Systems*, *Automotive Software Development*, *Automotive HMI*, *In-Vehicle Infotainment*, *Automotive Multimedia*, and *Automotive Camera Systems*. Essential infrastructure is provided by **Embedded & Operating Systems** (*Embedded Systems*, *Operating Systems*) and **Graphics & Multimedia** (*Graphics Frameworks*, *Multimedia*, *Image Processing*). **Connectivity & Data** encompasses protocols and data handling layers like *Automotive Networking*, *Communication Protocols*, *Automotive Data Processing*, *Automotive Data Storage*, and *Automotive Video Transmission*. Finally, **Tools & Security** provides necessary utilities and security mechanisms.

Table 4. List of Source Repositories by Category

Category	Repository Name
AD Algorithms	Eigen
AD Machine Learning	pytorch/pytorch tensorflow/tensorflow
AD Perception	PointCloudLibrary/pcl
AD Simulation	osrf/gazebo
Automotive Camera Systems	Omnivision OV5640 Driver

Continued on next page

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 4 – continued from previous page

Category	Repository Name
Automotive Data Processing	google/flatbuffers jbeder/yaml-cpp
Automotive Data Storage	facebook/rocksdb google/leveldb
Automotive HMI	Cairo HMI Studio Mir Pango Vulkan grimfang4/sdl2_gfx harfbuzz/harfbuzz memononen/nanovg ocornut/imgui wjakob/nanogui
Automotive Multimedia	libsdl-org/SDL
Automotive Networking	feaser/j1939stack
Automotive Operating Systems	AGL (Automotive Grade Linux)
Automotive Software Development	eclipse/kuksa.val
Automotive Video Processing	TI TVP5158 Driver
Automotive Video Transmission	Maxim MAX9286/MAX9288 Driver
Autonomous Driving	ROS (Robot Operating System) autowarefoundation/autoware
Communication Protocols	BlueZ CANopenNode/CANopenNode CW-B-W/ESP32-openLIN D-Bus MaJerle/stm32-usart-uart-dma-rx-tx Wpa_supplicant json-c/json-c openxc/uds-c protocolbuffers/protobuf Lwip adb ambrop72/badvpn curl/curl eclipse-mosquitto/mosquitto libimobiledevice/libimobiledevice libimobiledevice/libplist libimobiledevice/libusbmuxd libusb/libusb linux-can/can-utils mdns open-source-parsers/jsoncpp pupnp/pupnp tihmstar/usbmuxd2
Embedded Systems	Bosch ETAS RTA-OS NXP S32 SDK TI Sitara SDK Xilinx Vivado SDK ADI Blackfin SDK INTEGRITY RTOS
Graphics Frameworks	Freetype OpenGLES Qt for Automotive

Continued on next page

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 4 – continued from previous page

Category	Repository Name
	Wayland lvgl/lvgl zlgopen/awtk EGL
Image Processing	opencv/opencv
In-Vehicle Infotainment	Genivi WebPlatformForEmbedded/WPEWebKit raysan5/raylib
Multimedia	FFmpeg/FFmpeg GStreamer mstorsjo/fdk-aac tinyalsa/tinyalsa
Operating Systems	FreeRTOS/FreeRTOS-Kernel Linux Kernel QNX Neutrino RTOS STMicroelectronics/STM32CubeF4 U-Boot RT-Thread/rt-thread AOSP
Security	openssl/openssl
Storage	SQLite redis/redis
Tools	google/breakpad nayuki/QR-Code-generator

B.1.2. Source Repositories for Java

The Java repositories are organized into three primary categories based on their functionality and relevance to the automotive domain. **Automotive-Specific Applications** contains applications and services directly developed for vehicle functions, such as dashboards, instrument clusters, and vehicle monitoring systems. **System UI & Infrastructure** includes core Android system components adapted for automotive use, such as launchers, settings, and system updates. Finally, **Infotainment & Productivity** comprises applications focused on media playback, communication, and user productivity within the vehicle environment.

Table 5. List of Java Source Repositories by Category

Category	Repository Name
Automotive-Specific Applications	Android-Automotive-Info-App AndroidCarTracker CarServerArch MusiCar OpenXC-Caravan Realtime-Vehicle-Tracking UberCarAnimation Vehicle-In-Motion Vehicle-Monitoring-And-Routing-System VehicleApp aosp_platform_packages_apps_Car_SystemUI flink-vehicle-telematics openxc-android volvo-automotive-media-sample AndrOBD

Continued on next page

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 5 – continued from previous page

Category	Repository Name
	Cluster
	Hvac
	MultiDisplay
	Navigation-System-Instrument-Cluster
	Radio
	RotaryController
	SystemUISystembarsAAOS
	adas-dashboard-system
	simple_navigation
	SpeedoLimit
	TaxiAndroidOpen-Morocco
	WattsNearby
	mafscaling-2.7.1
	Dialer
	LensPicker
	LinkViewer
	LocalMediaPlayer
	Media
	Messenger
	Stream
	relaxing-sounds
System UI & Infrastructure	Android-SystemUI
	CompanionDeviceSupport
	DataSubscriptionReference
	Launcher
	Notification
	Overview
	Provision
	References
	Settings
	SettingsIntelligence
	SystemUI
	SystemUpdater
	Templates
	moselle

B.2. Static Analysis Rules for Java

The construction of the Java-based Intelligent Cockpit benchmark utilizes a *Tree-sitter based Static Analysis Protocol*. This approach addresses the specific challenges of the Android Automotive OS (AAOS) ecosystem, where domain knowledge is often implicitly embedded in framework inheritance and callback structures rather than explicit procedural calls.

B.2.1. Global FQCN Index Construction

To resolve dependencies without requiring a full Android build environment, we implement a project-wide indexing mechanism. We utilize the `tree-sitter-java` parser to scan all source files in the repository. The analyzer extracts the package declaration and top-level class definitions from the concrete syntax tree (CST).

By concatenating the package name and class name, we generate a Fully Qualified Class Name (FQCN) for every internal class. These are stored in a Global FQCN Index that maps each unique class identifier to its physical file path ($FQCN \rightarrow FilePath$). This index serves as the ground truth for project-internal definitions.

B.2.2. Dependency Tracing and Filtering

For each target file, the analysis pipeline identifies potential cross-file contexts through two primary mechanisms:

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

1. **Explicit Import Resolution**: The analyzer parses import statements and checks them against the Global FQCN Index. Imports that match an entry in the index are flagged as project-specific dependencies.

2. **Implicit Same-Package References**: The analyzer identifies identifiers that are undefined locally but match class names existing within the same package in the Global Index.

Crucially, we apply a strict filtering rule to distinguish between domain logic and generic Android usage. External dependencies, such as standard SDK classes (e.g., `android.content.Context`), are excluded. Only dependencies that resolve to a file within the curated repository are retained, ensuring the benchmark evaluates the model's understanding of the specific vehicle service implementations (e.g., `CarService` or `VehiclePropertyIds`).

B.3. Static Analysis Rules for C

For the C-based Vehicle Middleware benchmark, we employ a `libclang`-based Symbol Resolution pipeline. This method is chosen to overcome the limitations of regex-based parsing, which cannot accurately resolve complex preprocessor macros or hardware-specific definitions common in embedded C.

B.3.1. Global Symbol Table Construction

We construct a Global Symbol Table by traversing the Abstract Syntax Tree (AST) of all header files (`.h`) in the dataset. Using `libclang`'s cursor visitation capabilities, we extract the signatures of:

- Function declarations (API surface).
- Struct and Union definitions (Data layouts).
- Typedefs (Opaque types).
- Macro definitions (Hardware registers and constants).

This AST-level extraction allows us to capture hardware-mapped macros—often defined via complex preprocessor directives—which constitute a significant portion of automotive middleware logic.

B.3.2. Usage Tracking and Domain Relevance Filtering

To generate context-aware tasks, we parse the target source files (`.c`) to identify identifiers that are used but not defined within the file scope. These undefined identifiers are cross-referenced against the Global Symbol Table.

A critical component of this pipeline is the Domain Relevance Filter. Mere cross-file usage is insufficient; the dependency must pertain to critical system definitions. We filter the resolved symbols based on automotive-specific nomenclature to ensure high-value retrieval targets. Dependencies are retained only if they relate to:

- **Hardware Abstractions**: Symbols involving HAL, drivers, BSP (Board Support Package), or register maps.
- **Communication Protocols**: Symbols related to vehicle buses such as CAN, SPI, I2C, or UART.
- **Real-time Signals**: Symbols denoting sensors, actuators, or signal processing logic.

This filtering ensures that the constructed benchmark strictly evaluates the model's capability to reason about the vehicle's hardware-software interface, filtering out generic C utility functions that do not require specialized domain knowledge.

B.4. Java Dataset Construction and Filtering

The Java dataset focuses on the Intelligent Cockpit Application Layer, specifically targeting the Android Automotive OS (AAOS) ecosystem. Unlike general Android applications, these projects exhibit high coupling with vehicle-specific properties.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

B.4.1. Context-Aware Task Construction via Tree-sitter

To accurately capture the non-local domain knowledge without the computational overhead of a full Android build environment, we implemented a Tree-sitter based Static Analysis Protocol . This approach avoids mock compilation and focuses on resolving cross-file dependencies through structural parsing:

1. **FQCN Indexing:** We parse all Java source files using `tree-sitter-java` to extract class definitions. A project-wide index is constructed, mapping every Fully Qualified Class Name (FQCN) to its specific file path.

2. **Import Resolution:** When processing a target file, we analyze its import declarations against the FQCN index. This allows us to strictly distinguish between *project-internal* dependencies (which require context retrieval) and *external* dependencies (e.g., standard Android SDK classes like `android.content.Context`).

3. **Context Extraction:** Based on the resolved internal imports, we identify the specific cross-file classes and methods required for the current completion target, effectively modeling the decentralized storage of domain knowledge in framework-heavy architectures.

B.4.2. Heuristic-based Filtering Strategy

Following the context extraction, we apply a rigorous filtering strategy to remove low-value samples. As detailed in Phase 3 of our pipeline, simple "import" or "package" completions do not reflect the complex logic required for SDV development. We implemented a filtering script that combines regular expressions with "strong keep" signals to preserve logical depth.

Regular Expressions for Low-Value Identification. We defined specific regular expressions to detect lines that are likely 'import' statements or 'package' declarations, as well as simple ground truths often associated with import completions.

```
# Regex to identify lines starting with 'import'
IMPORT_LINE_RE = re.compile(r"^(import\s+[\w\.\*]+)$")

# Regex to identify lines starting with 'package'
PACKAGE_LINE_RE = re.compile(r"^(package\s+[\w\.]+;?)$")

# Regex to identify simple ground truth statements
# (e.g., "List;" or "java.util.List;") often found in import completions
SIMPLE_STMT_RE = re.compile(r"^[A-Za-z_][\w\.]*\s*;\s*$")
```

Filtering Logic with Strong Keep Signals. To prevent the accidental removal of legitimate code that resembles import structures, we inspect the last 400 characters of the input context. If any "Strong Keep Signal" (e.g., control flow keywords like `if`, `for`, or definitions like `class`, `@Override`) is found, the sample is forcibly retained.

The final filtering decision logic is implemented as follows:

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
def is_import_completion(input_text: str, gt: str) -> bool:
    last = last_nonempty_line(input_text)

    # 1) If the last line is an import statement
    if last.startswith("import "):
        # If GT is a simple class name/path or ends with ';', likely an import
        if SIMPLE_STMT_RE.match(gt.strip()) or gt.strip().endswith(";"):
            return True
        return True

    # 2) If the last line is a package declaration
    if last.startswith("package "):
        return True

    return False

def has_strong_keep_signal(input_text: str) -> bool:
    # Check the last 400 characters for logic keywords
    tail = input_text[-400:]
    return any(h in tail for h in KEEP_HINTS)
```

B.5. C Language Dataset Construction and Filtering

The construction of the C language benchmark targets the Vehicle Middleware and Hardware Abstraction Layer (HAL). This domain operates under strict hardware constraints and relies heavily on proprietary macros and interface definitions.

B.5.1. Domain-Specific Data Curation

We performed a targeted curation process to identify repositories representative of the safety-critical layer. General-purpose C repositories (e.g., web servers) were excluded. Instead, we utilized domain-specific keywords such as *VHAL*, *ECU*, *CAN bus*, and *SOME/IP*.

Following rigorous manual review by domain experts, we retained 22 high-quality repositories that capture the essential vehicle hardware abstractions. These repositories can be categorized into five key domains relevant to SDV:

1. **Embedded Systems & RTOS:** Includes kernels like `FreeRTOS-Kernel` and hardware drivers (e.g., `stm32-usart`), representing the deterministic control layer.
2. **Networking & Communication:** Covers automotive connectivity stacks, including `lwip` (TCP/IP), `mosquitto` (MQTT for telemetry), and secure communication libraries.
3. **Multimedia & HMI:** Comprises libraries for in-vehicle infotainment, such as `lvgl` for embedded GUIs and `FFmpeg` for media processing.
4. **Perception & Autonomy:** Includes fundamental libraries for ADAS, such as `opencv` (computer vision) and `pcl` (LiDAR point clouds).
5. **Data Infrastructure:** Features high-performance serialization and storage optimized for embedded environments (e.g., `protobuf`, `leveldb`).

B.5.2. Libclang-based Symbol Resolution

Given the prevalence of macro-heavy definitions and lack of unified build systems in embedded C, we addressed the task construction challenge via a Libclang-based Symbol Resolution pipeline:

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

- **Global Symbol Table Construction:** We parse all header files (`.h`) using libclang's AST traversal to extract signatures of functions, structs, typedefs, and hardware-mapped macros. This captures definitions often hidden behind complex preprocessor directives.

- **Usage Tracking Domain Filtering:** We analyze source files (`.c`) to identify identifiers undefined locally but present in the Global Symbol Table. A domain filter is applied to retain only dependencies involving critical system definitions (e.g., hardware register macros), ensuring the benchmark tests the model's navigation of specific vehicle middleware vocabulary rather than generic syntax.

B.6. Benchmark Evaluation

This section provides an analytical evaluation of the constructed benchmark from multiple complementary perspectives, aiming to characterize its domain specificity, structural difficulty, and semantic concentration on automotive-specific software components. All analyses are based on the proposed AAOS score, which measures the degree to which a code artifact is entangled with Android Automotive OS (AAOS)–specific APIs and symbols.

Package-level Analysis. Figure 8 presents the top-ranked packages by AAOS score. We observe that system-level and vehicle-facing packages such as `Settings`, `Hvac`, `Dialer`, and `SystemUpdater` dominate the ranking. These packages are central to in-vehicle user interaction and system control, and typically contain dense dependencies on automotive middleware APIs. Notably, high AAOS scores are often accompanied by multiple unique files per package, indicating that automotive-specific knowledge is distributed across files rather than isolated to a single implementation unit. This property makes completion tasks particularly challenging for models relying on shallow lexical cues.

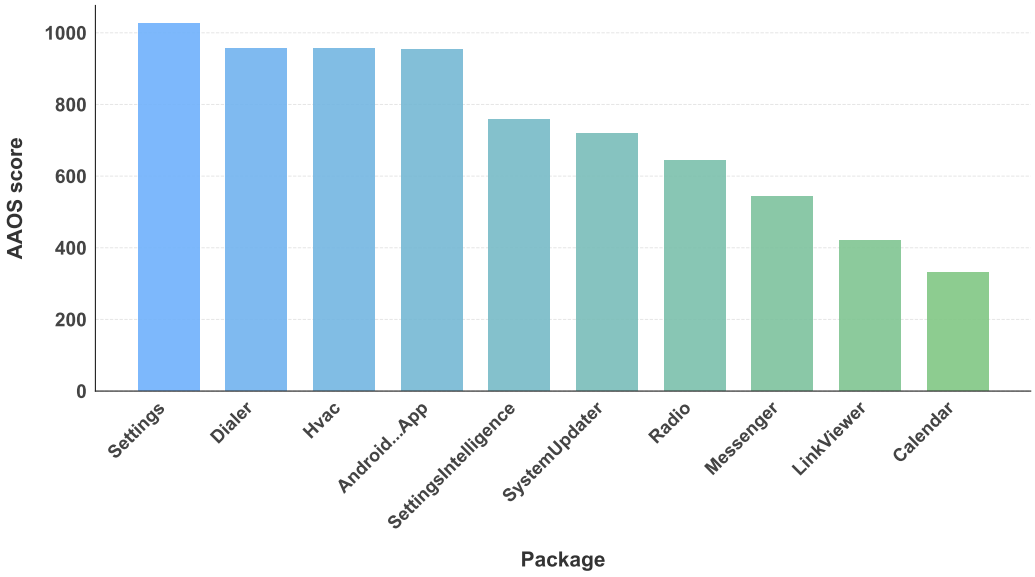


Figure 8. Top-10 packages by AAOS score

File-level Analysis. At the file level, Figure 9 shows that the highest-scoring files correspond to concrete vehicle interaction logic, such as HVAC control (`HvacPanelController.java`, `HvacController.java`) and car-oriented UI components (`CarUiRecyclerView.java`, `CarSearchFragment.java`). These files are typically characterized by frequent interactions with vehicle properties, UI state constraints, and system services. Their elevated AAOS scores suggest that they represent "hard cases" where correct code completion requires precise understanding of AAOS-specific APIs rather than general Android patterns.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

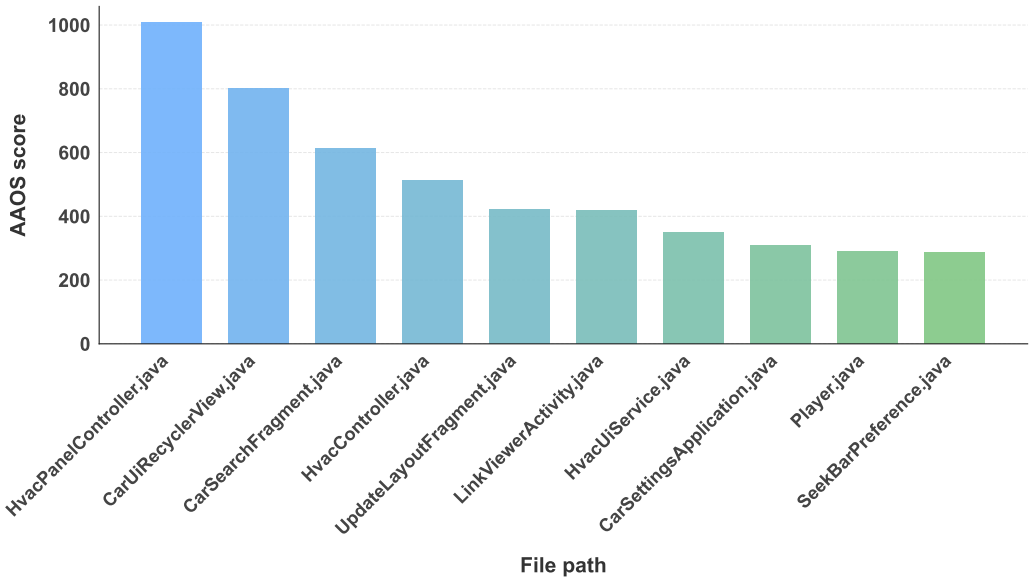


Figure 9. Top-10 files by AAOS score

Key Symbol Distribution in Code Bodies. Figure 10 reports the most frequent AAOS-related symbols appearing in code bodies. Symbols such as `Car`, `CarUiRecyclerView`, and `CarPropertyValue` dominate the distribution, revealing two strong signals: (1) a pronounced AAOS UI ecosystem centered around `CarUi*` abstractions, and (2) extensive use of vehicle property and VHAL-related entities (e.g., `CarPropertyValue`), which are tightly coupled with vehicle speed, gear state, HVAC, and lighting controls. These symbols are rarely observed in general-purpose Android corpora, reinforcing the domain exclusivity of the benchmark.

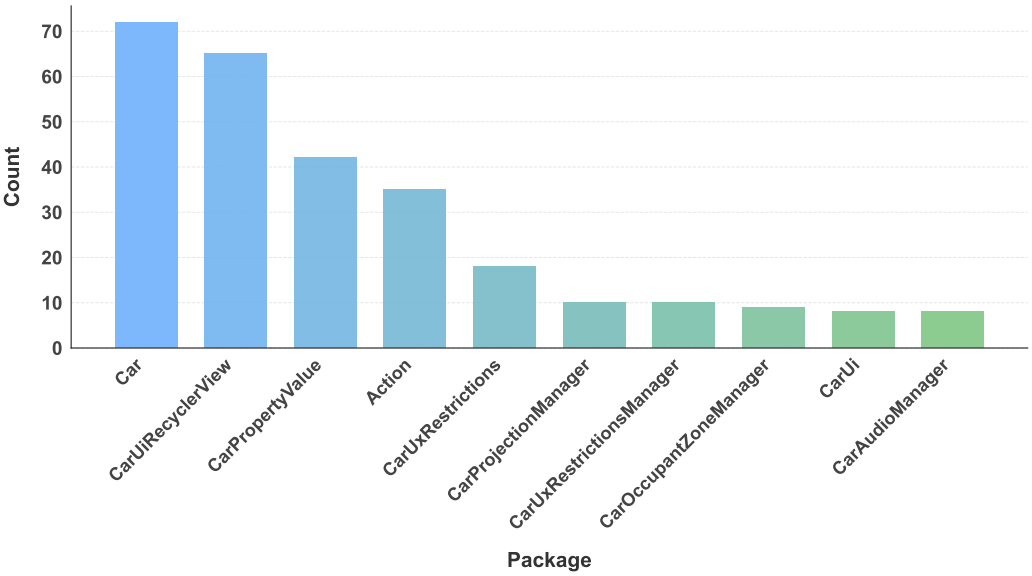


Figure 10. Top-10 AAOS key symbols (heuristic)

Explicit Dependency Analysis via Imports. To further validate that the benchmark captures explicit automotive dependencies rather than incidental naming overlap, we analyze fully-qualified import statements and their usage frequency. As shown in Figure 11, imports such as `android.car.Car`, `CarUxRestrictions`, and HVAC-related controllers appear frequently, reflecting direct coupling to AAOS system services and safety-related UX constraints.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

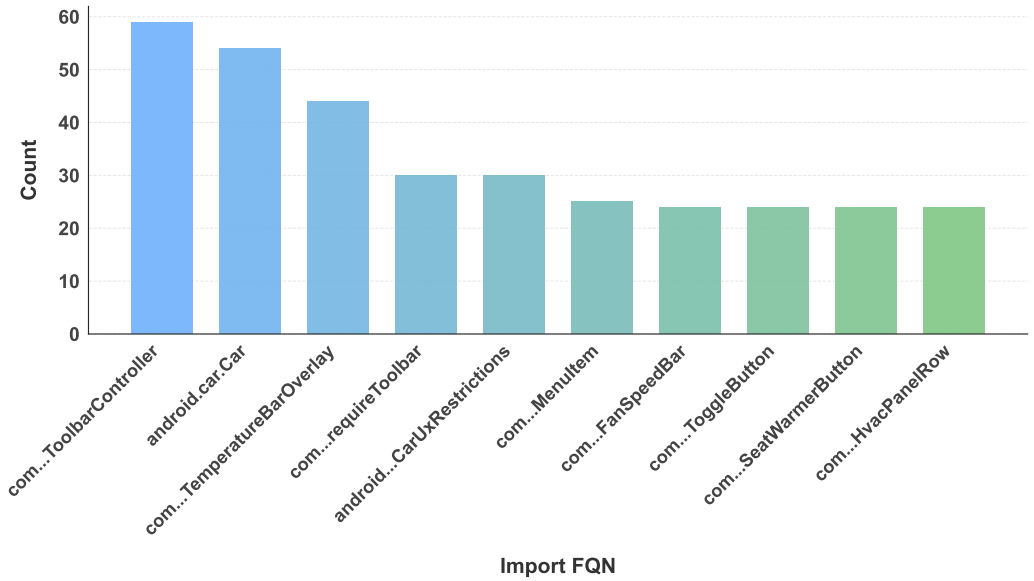


Figure 11. Top-10 AAOS import FQNs

Moreover, Figure 12 demonstrates that the imported classes (e.g., `CarHvacManager`, `VehicleAreaSeat`) are actively used within code bodies, confirming that these dependencies are semantically essential rather than superficial.

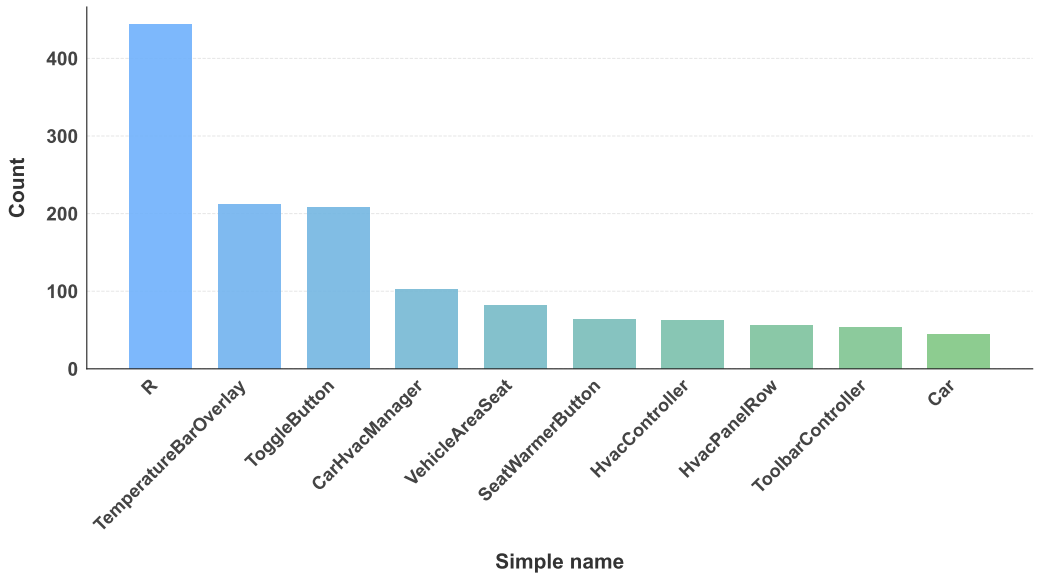


Figure 12. Top-10 imported AAOS simple-name occurrences

Implications for Benchmark Difficulty. Overall, this evaluation reveals that the benchmark is dominated by automotive-exclusive APIs, UI abstractions, and vehicle property interfaces, forming a long-tail distribution that is poorly covered by generic code models. Completion difficulty is therefore driven not by syntactic complexity, but by the requirement to correctly resolve cross-file, cross-package, and domain-specific dependencies. This makes the benchmark particularly suitable for assessing a model's ability to acquire and apply specialized automotive software knowledge, rather than memorizing common programming idioms.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

C. Extended Experimental Results

In this section, we provide detailed specifications regarding the experimental setup, including hardware configurations, model specifications, inference hyperparameters, and the prompt construction methodology used for evaluation.

C.1. Hardware and Environment

The experiments were conducted on a multi-GPU server environment. Specifically, we utilized 4 GPUs for inference, as indicated by the active CUDA device visibility (IDs 0, 1, 2, 3) and the vLLM tensor parallelism setting ($TP = 4$).

C.2. Inference Hyperparameters

For local open-source models (CodeLlama, DeepSeek, Qwen, etc.), we utilized the vLLM library for inference to optimize throughput. We employed greedy decoding (Temperature = 0) to ensure the reproducibility of our results. The specific inference parameters are detailed in Table 6.

Table 6. Inference Hyperparameters for Local Models.

PARAMETER	VALUE
TEMPERATURE	0.0
TOP-p	1.0
TOP-k	-1 (DISABLED)
MAX NEW TOKENS	48
REPETITION PENALTY	1.0 (NONE)
SAMPLING	FALSE (GREEDY)
TENSOR PARALLELISM	4

Note: Parameters derived from evaluation scripts and `utils.py`.

C.3. Retrieval and Prompt Construction

Unlike vector-based RAG systems, our approach utilizes a static analysis-based retrieval mechanism to construct the context window.

Retrieval Settings. The global configuration for context retrieval enforces specific constraints on the information extracted from the dependency graph for both C and Java datasets:

- `ONLY_DEF = True`: The retrieval system is configured to extract only the definitions (signatures) of functions, methods, classes, or structs from dependency files, excluding their implementation bodies to conserve context tokens.
- `ENABLE_DOCSTRING = True`: Documentation strings (such as JavaDocs for Java or comments for C) associated with the retrieved definitions are included to provide semantic information.
- **Sorting Strategy**: Retrieved context nodes are ordered using a pseudo-topological sort based on the dependency graph to respect compilation and logical dependencies.

Prompt Format. The final prompt constructed for the model varies slightly between languages to adhere to their respective syntax. Figure 13 and Figure 14 illustrate the prompt structures for C and Java, respectively.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
/*
[Retrieved Context (Topologically Sorted)]
// path/to/dependency_A.h
typedef struct { ... } A;

// path/to/dependency_B.h
void function_B(A* a);
*/

// path: <Target_File_Relative_Path>
<Input_Source_Code_Context>
```

Figure 13. Prompt structure for C evaluation. Context includes topologically sorted header definitions and struct declarations extracted via `node_prompt.py`.

```
/*
// <Dependency_File_Path>
<Dependency_Class_Definition>
<Dependency_Method_Signatures>
<Optional_Docstrings>

... (Additional Dependencies)
*/
// path: <Target_File_Relative_Path>
<Input_Source_Code_Context>
```

Figure 14. Prompt structure for Java evaluation. The context block contains signatures and docstrings extracted from dependency files via static analysis.

C.4. Prompt Engineering Specification

To evaluate the capabilities of the Large Language Model (LLM) in the context of repository-level code completion, we employed a strict zero-shot prompting strategy. The prompt design is bifurcated into a System Instruction (to enforce syntactic continuity and output constraints) and a User Context (incorporating Graph-based RAG retrieval results and the immediate code context).

System Instruction Schema. The system prompt is engineered to suppress the model's chat capabilities and force it into a deterministic "completion engine" mode. The directives are prioritized as follows:

System Prompt for GPT5

```
1. Role Definition:
   You are a dedicated code completion engine.

2. Continuity Constraint:
   You must complete the code starting exactly where the user input ends
   (i.e., at the cursor position).

3. Negative Constraint (Anti-Repetition):
   Do NOT repeat any part of the provided user input.
   Only output the completion suffix.

4. Output Formatting:
   Output only raw code.
   Do not include Markdown formatting, explanations, or conversational text.

5. Granularity Control:
   Generate only the immediate next statement or a minimal logical code block.
```

User Input Construction. The user prompt aggregates global context (retrieved via the Graph-RAG module) and local context (the target

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

file). The structure is organized to prioritize local syntactic distinctness while providing necessary cross-file dependencies. Figure 15 illustrates the user prompt template.

User Input Construction for GPT5

```
[SECTION 1: RETRIEVED CONTEXT]
// The following references are retrieved from the project dependency graph
// Reference 1: Definition of <Relevant_Class_Or_Function>
<Code_Snippet_1>

// Reference 2: Usage in <Related_File>
<Code_Snippet_2>

...

[SECTION 2: TARGET CONTEXT]
// The current file being edited
<Target_File_Path>
<Preceding_Code_Context>
// ... (Code leading up to the cursor) ...
<Immediate_Preceding_Lines>  <-- Cursor Position
```

Figure 15. User Prompt Template utilized for generation. The model is expected to generate the code logically following `<Immediate_Preceding_Lines>` based on the constraints defined in the System Prompt.

D. Qualitative Analysis & Case Studies

In this section, we present a comprehensive qualitative analysis of the model's generation failures. To capture language-specific nuances and the distinct challenges posed by different programming paradigms, we divide our error taxonomy into two primary categories. **Types A through G** characterize failures observed in the Java dataset, specifically within the Android application context, highlighting issues related to object-oriented frameworks and API contracts. In contrast, **Types H through K** address specific failure modes identified in the C language dataset, focusing on the complexities of procedural programming and low-level system interactions. The following subsections detail these categories with representative case studies.

D.1. Type A: Interface and Signature Mismatch

Definition: The model generates API names or method signatures that are semantically plausible but legally incorrect according to the strict interface contracts or library specifications.

Root Cause Analysis:

- **Symbol Resolution Failure:** The model treats method naming as a semantic generation task rather than a strict symbol retrieval task. It prioritizes "readable" names over exact matches with existing abstract classes or interfaces.

- **API Safety Neglect:** The model fails to treat the API contract as a hard constraint, leading to compilation errors despite semantic correctness.

Case Studies:

Case ID 96 (API Hallucination): The model imports `TelephonyManager` based on the semantic context of phone numbers, whereas the ground truth requires the utility class `PhoneNumberUtils`.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
  {
    "id": 96,
    "pkg": "Calendar",
    "fpath": "src/com/android/car/settings/common/SettingsFragment.java",
    "input":"...
        import android.content.pm.PackageManager;
        import android.net.Uri;
        import android.telephony.",
    "raw_res": "TelephonyManager;",
    "prompt_res": "PhoneNumberUtils;",
    "gt": "PhoneNumberUtils;"
  }
```

Case ID 1757 (Signature Mismatch): In an `Override` scenario, the model generates the parameter name `eventBuilder`, differing from the parent class's strictly defined `logEventBuilder`.

```
{
    "id": 1757,
    "pkg": "Templates",
    "fpath": "Host/app/src/main/java/com/android/car/templates/host/di
        /TelemetryHandlerStub.java",
    "input": "...
        public void logCarAppTelemetry(TelemetryEvent.",
    "raw_res": "Builder eventBuilder) {",
    "prompt_res": "Builder logEventBuilder) {",
    "gt": "Builder logEventBuilder) {"
  }
```

Case ID 1722 (Strict Naming): The model hallucinates `getAppComponentName()`, which is semantically identical but syntactically distinct from the correct AutoValue/Interface method `getComponentName()`.

```
  {
    "id": 1722,
    "pkg": "Templates",
    "fpath": "app/src/main/java/com/davidmagalhaes/androidcartracker
        /YourService.java",
    "input": "...
        public abstract Optional<CarAppApi> getCarAppApi();
        public abstract Optional<",
    "raw_res": "ComponentName> getAppComponentName();",
    "prompt_res": "ComponentName> getComponentName();",
    "gt": "ComponentName> getComponentName();"
  }
```

D.2. Type B: Structural Context Misinterpretation

Definition: The model misidentifies the syntactic scope of the current cursor position, confusing member declaration blocks with method body logic or static initialization blocks.

Root Cause Analysis:

- **Local Structure Boundary Failure:** The model lacks a global view of the class structure, failing to distinguish between where fields should be defined and where complex logic should be executed.

Case Studies:

Case ID 2662 (Logic vs. Configuration): Following an `@Override` annotation, the ground truth expects a configuration getter (`getServicePkgName`). The model instead attempts to hallucinate a complex business logic method (`createService`).

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
{
  "id": 2662,
  "pkg": "CarServerArch",
  "fpath": "SDK/src/main/java/com/fwk/sdk/hvac/HvacManager.java",
  "input": "...
      public static HvacManager getInstance() {
      if (sHvacManager == null) {
          synchronized (HvacManager.class) {
              if (sHvacManager == null) {
                  sHvacManager = new HvacManager();
                  }
              }
          }
      return sHvacManager;
      }    @",
  "raw_res": "Override\n
      protected HvacInterface createService(IBinder binder) {",
  "prompt_res": "Override\n  protected String getServicePkgName() {",
  "gt": "Override\n    protected String getServicePkgName() {"
}
```

Case ID 585 (Declaration vs. Initialization): In a variable declaration context (expecting `Context mContext`), the model attempts to generate a complex static array initialization (`ALL_AUDIO_PROJECTION`), misjudging the class member scope.

```
{
  "id": 585,
  "pkg": "LocalMediaPlayer",
  "fpath": "src/com/android/car/media/localmediaplayer/DataModel.java",
  "input": "...
      public static final String PATH_KEY = \"PATH\";
      private ",
  "raw_res":
      "static final String[] ALL_AUDIO_PROJECTION = new String[] {",
  "prompt_res": "Context mContext;",
  "gt": "Context mContext;"
}
```

D.3. Type C: Context-Agnostic Variable Typing

Definition: The model selects variable types based on lexical associations (e.g., recent imports or file names) rather than the logical role the variable plays in the code or the class inheritance structure.

Root Cause Analysis:

- **Lexical Bias:** The model is misled by local imports or token probability (e.g., seeing "Activity" in imports and using it as a type) rather than inferring the type from usage.

- **Inheritance Blindness:** The model ignores the class definition (e.g., `extends Service`) when suggesting context types.

Case Studies:

Case ID 1772 (Inheritance Ignorance): Inside a file named `YourService.java` (extending Service), the model incorrectly types a variable as `Activity`, ignoring the file's semantic role.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
{
  "id": 1772,
  "pkg": "AndroidCarTracker",
  "fpath": "app/src/main/java/com/davidmagalhaes/androidcartracker
      /YourService.java",
  "input": "package com.davidmagalhaes.androidcartracker;\n\n
      import android.app.",
  "raw_res": "Activity;",
  "prompt_res": "Service;",
  "gt": "Service;"
}
```


Case ID 1119 (Type Confusion): The model imports `drawable.Drawable` when the method signature requires a geometric `Rect`.


```
{
  "id": 1119,
  "pkg": "RotaryController",
  "fpath": "src/com/android/car/rotary/DumpUtils.java",
  "input": "...
      import android.content.ComponentName;\n
      import android.graphics.",
  "raw_res": "drawable.Drawable;",
  "prompt_res": "Rect;",
  "gt": "Rect;"
}
```


Case ID 259 (Context Misjudgment): In a `BootCompleteReceiver`, the model defaults to importing `Context` instead of the necessary `BroadcastReceiver`.


```
{
  "id": 259,
  "pkg": "Dialer",
  "fpath": "src/com/android/car/hvac/BootCompleteReceiver.java",
  "input": "...
      package com.android.car.hvac;\n\n
      import android.content.",
  "raw_res": "Context;",
  "prompt_res": "BroadcastReceiver;",
  "gt": "BroadcastReceiver;"
}
```


D.4. Type D: Incomplete Control Flow & Error Handling

Definition: The model identifies error conditions or callbacks but fails to generate the critical control flow statements (e.g., `finish()`, `return`, or correct callback invocation) required for program correctness.

Root Cause Analysis:

- **Intent Modeling Deficit:** The model recognizes the "error state" but fails to model the "remedial action", treating the completion as a logging task rather than a control flow task.

Case Studies:

Case ID 555 (Missing Termination): Upon detecting a null URL, the model generates a log statement but fails to call `finish()`, leaving the Activity in an invalid state.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
  {
    "id": 555,
    "pkg": "LinkViewer",
    "fpath": "src/com/android/car/linkviewer/LinkViewerActivity.java",
    "input": "...
        String url = getUrlFromIntent(getIntent());\n
        if (url == null) {\n ",
    "raw_res": "Log.e(TAG, \"No URL found in intent\");",
    "prompt_res": "finish();",
    "gt": "finish();"
  }
```

Case ID 2654 (Incorrect Callback Invocation): The model attempts to pass arguments to a callback (`onServiceConnected`) that is defined as void/parameter-less in the interface context.

```
  {
    "id": 2654,
    "pkg": "CarServerArch",
    "fpath": "SDK/src/main/java/com/fwk/sdk/base/BaseConnectManager.java",
    "input": "...
        if (mServiceListener != null) {\n
            mServiceListener.",
    "raw_res": "onServiceConnected(mProxy);",
    "prompt_res": "onServiceConnected();",
    "gt": "onServiceConnected();"
  }
```

D.5. Type E: Test Framework & Domain Confusion

Definition: When operating within test directories (e.g., `tests/multivalent`), the model conflates syntax and assertions from different testing frameworks (JUnit, Mockito, Truth), violating the project's testing dialect.

Root Cause Analysis:

- **Context Domain Failure:** The model fails to recognize the specific "Test Scope" constraints.
- **Consistency Inference Gap:** It lacks the ability to infer framework consistency (e.g., "If Mockito is used here, Truth should likely be used there").

Case Studies:

Case ID 1336 (Framework Mismatch): The code context implies Mockito usage, but the model imports JUnit's `Assert.assertEquals`.

```
  {
    "id": 1336,
    "pkg": "Settings",
    "fpath": "tests/multivalent/src/com/android/car/settings/enterprise/Base
AdminGrantedPermissionsApplicationListPreferenceControllerTestCase.java",
    "input": "...
        import static com.google.common.truth.Truth.assertWithMessage;\n\n",
    "raw_res": "import static org.junit.Assert.assertEquals;",
    "prompt_res": "import static org.mockito.ArgumentMatchers.any;",
    "gt": "import static org.mockito.ArgumentMatchers.any;"
  }
```

Case ID 1354 (Assertion Library Mismatch): The ground truth requires the Google Truth library (`assertThat`), but the model provides a generic Android context import.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
  {
    "id": 1354,
    "pkg": "Settings",
    "fpath": "tests/multivalent/src/com/android/car/settings/enterprise
    /BaseEnterprisePreferenceControllerTestCase.java",
    "input": "...
        package com.android.car.settings.enterprise;\n\n",
    "raw_res": "import android.content.Context;",
    "prompt_res": "import static com.google.common.truth.Truth.assertThat;",
    "gt": "import static com.google.common.truth.Truth.assertThat;"
  }
```


D.6. Type F: Symbolic Hallucination in Constants

Definition: The model invents constant names based on linguistic patterns (e.g., antonyms like Visible/Invisible) rather than retrieving the exact, often asymmetrical, constants defined in the AOSP codebase.

Root Cause Analysis:

- **Probabilistic Generation vs. Fact Recall:** The model generates tokens based on high probability word associations rather than accessing exact knowledge of system-level definitions.

Case Studies:

Case ID 1557 (Lifecycle Hallucination): The model generates `USER_LIFECYCLE_EVENT_TYPE_VISIBLE` (likely inferring from "Invisible"), whereas the correct constant is `..._REMOVED`.


```
  {
    "id": 1557,
    "pkg": "SystemUI",
    "fpath": "src/com/android/systemui/car/userpicker
    /UserEventManager.java",
    "input": "...
        package com.android.systemui.car.userpicker;\n\n
        import static
android.car.user.CarUserManager.USER_LIFECYCLE_EVENT_TYPE_CREATED;\n
        import static
android.car.user.CarUserManager.USER_LIFECYCLE_EVENT_TYPE_INVISIBLE;\n",
    "raw_res": "import static
android.car.user.CarUserManager.USER_LIFECYCLE_EVENT_TYPE_VISIBLE;",
    "prompt_res": "import static
android.car.user.CarUserManager.USER_LIFECYCLE_EVENT_TYPE_REMOVED;",
    "gt": "import static
android.car.user.CarUserManager.USER_LIFECYCLE_EVENT_TYPE_REMOVED;"
  }
```


Case ID 1605 (System Symbol Hallucination): The model imports `SparseArray` (a common utility) when the specific system-level logging tool `Slog` is required.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
{
  "id": 1605,
  "pkg": "SystemUI",
  "fpath": "src/com/android/systemui/car/userpicker
  /UserEventManager.java",
  "input": "...
      import android.os.Looper;\n
      import android.os.UserHandle;\n
      import android.os.UserManager;\n
      import android.util.Log;\n
      import android.util.",
  "raw_res": "SparseArray;",
  "prompt_res": "Slog;",
  "gt": "Slog;"
}
```


D.7. Type G: Generic Type Erasure & Simplification

Definition: When completing complex generic signatures, the model tends to "simplify" types to `String`, ignoring semantic cues that suggest numeric or custom types are required.

Root Cause Analysis:

- **Shallow Type Inference:** The model treats generic parameters as placeholders to be filled with the most common type (`String`) rather than inferring the data type from the variable name (e.g., "Speed" implies numerical data).

Case Studies:

Case ID 2788 (Type Simplification): For a tuple representing `AverageSpeedKey`, the model generates `Tuple3<String, String, String>`, whereas the ground truth semantically requires `Tuple3<String, Integer, Integer>` to store speed and direction data.


```
{
  "id": 2788,
  "pkg": "flink-vehicle-telematics",
  "fpath": "flink-vehicle-telematics-master/src/main/java/fediazgon/
  AverageSpeedControl.java",
  "input": "...
      private static class AvgSpeedKey extends ",
  "raw_res": "Tuple3<String, String, String> {",
  "prompt_res": "Tuple3<String, Integer, Integer> {",
  "gt": "Tuple3<String, Integer, Integer> {"
}
```


D.8. Type H: Function Signature Hallucination

Definition: When completing static function prototypes in driver development, the model hallucinates parameter types based on lexical similarity in the function name, ignoring the strict type definitions required by the hardware abstraction layer (HAL).

Root Cause Analysis:

- **Lexical vs. Semantic Association:** The model associates keywords (e.g., "PLL") in the function name with the most textually similar struct type, rather than retrieving the semantically correct configuration struct defined in the header files.
- **Weak Symbol Indexing:** The RAG system relies on text similarity, failing to resolve the exact cross-file symbol dependencies required for accurate function declarations.

Case Studies:

Case ID 2841 (Type Confusion): In an STM32 driver context, the model incorrectly infers `LL_UTILS_PLLInitTypeDef` as the parameter for a system switch function, whereas the ground truth requires `LL_UTILS_ClkInitTypeDef` and a frequency integer.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
  {
    "id": 2841,
    "pkg": "stm32-usart-uart-dma-rx-tx",
    "fpath": "drivers/STM32F7xx_HAL_Driver/Src/stm32f7xx_ll_utils.c",
    "input": "...
        static ErrorStatus UTILS_SetFlashLatency(uint32_t HCLK_Frequency);
        static ",
    "raw_res": "ErrorStatus UTILS_EnablePLLAndSwitchSystem
(LL_UTILS_PLLInitTypeDef *UTILS_PLLInitStruct,\n
        uint32_t UTIL",
    "prompt_res": "ErrorStatus UTILS_EnablePLLAndSwitchSystem
(uint32_t SYSCLK_Frequency, LL_UTILS_ClkInitTypeDef *UTILS_ClkInitStruct);",
    "gt": "ErrorStatus UTILS_EnablePLLAndSwitchSystem
(uint32_t SYSCLK_Frequency, LL_UTILS_ClkInitTypeDef *UTILS_ClkInitStruct);"
  }
```

D.9. Type I: Logical Context Discontinuity

Definition: The model fails to complete immediate, low-level logic (such as bitwise register operations) implied by comments, skipping ahead to subsequent procedural function calls.

Root Cause Analysis:

- **Coarse-Grained Retrieval:** The retrieval mechanism likely fetches the subsequent function definition block, causing the model to "look ahead" too aggressively and miss the fine-grained, line-level logic required by the current cursor position.
- **Instruction Misalignment:** The model ignores explicit comment directives (e.g., "and SysTick") that demand specific register manipulation.

Case Studies:

Case ID 630 (Logic Skipping): In a FreeRTOS porting scenario, the model ignores an incomplete bitwise OR operation intended to set the SysTick priority, jumping immediately to a timer setup function call instead.

```
  {
    "id": 630,
    "pkg": "FreeRTOS-Kernel",
    "fpath": "portable/CCS/ARM_CM3/port.c",
    "input": "...
        portNVIC_SHPR3_REG |= portNVIC_PENDSV_PRI;\n    ",
    "raw_res": "/* Start the timer that generates the tick ISR.
Interrupts are disabled \n
        * here already. */\n
        vPortSetupTimerInterrupt();",
    "prompt_res": "portNVIC_SHPR3_REG |= portNVIC_SYSTICK_PRI;",
    "gt": "portNVIC_SHPR3_REG |= portNVIC_SYSTICK_PRI;\n"
  }
```

D.10. Type J: Domain-Specific Argument Overfitting

Definition: When generating function arguments for specific libraries, the model reverts to generic C programming patterns (e.g., offset/length pairs) instead of using the domain-specific flags or variables defined by the protocol stack.

Root Cause Analysis:

- **Training Data Bias:** The model's training on vast amounts of generic C code overpowers the local context, leading it to predict standard "Length/Offset" patterns even when the specific library requires boolean flags or checksums.
- **Local Variable Distraction:** Preceding variable names (e.g., `http_hdr_len`) prime the model to generate related generic terms rather than following the specific API definition.

Case Studies:

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Case ID 755 (Generic Hallucination): For an LwIP `makefsdata` utility function, the model generates generic `offset` and `length` arguments, while the ground truth requires specific `chksum` pointers and compression flags.

```
  {
    "id": 755,
    "pkg": "badvpn",
    "fpath": "lwip/src/apps/httpd/makefsdata/makefsdata.c",
    "input": "...
        int file_write_http_header(FILE *data_file, const char *filename,
int file_size, u16_t *http_hdr_len,\n
        ",
    "raw_res": "u16_t *http_hdr_offset,
u16_t *http_data_offset, u16_t *http_data_len);",
    "prompt_res": "u16_t *http_hdr_chksum,
u8_t provide_content_len, int is_compressed);",
    "gt": "u16_t *http_hdr_chksum,
u8_t provide_content_len, int is_compressed);\n"
  }
```

E. Implementation Details of RAG

In alignment with the **Graph-RAG** approach described in Section 4, we implement a reference system designed to capture the heterogeneous structure of intelligent cockpit software. The implementation details, including parsing strategies, ambiguity resolution, and safety-aligned prompt formulation, are described below.

E.1. Structure-Aware Graph Construction

The graph construction pipeline is implemented to handle the specific topologies of C Middleware and Java Application layers using a polyglot parsing strategy.

Polyglot Parsing Pipeline. The system employs distinct parsers to extract structural entities and strictly modeled relationships:

- **C Middleware**: We utilize the Python bindings for `libclang` to traverse the Abstract Syntax Tree (AST) of C/C++ files. This parser extracts high-level entities such as `Function`, `Struct`, and `Macro` definitions, rather than raw text tokens. It explicitly captures `#include` directives as `INCLUDE` edges and variable types as `TYPEOF` edges, physically linking usage to definition.
- **Java Application**: We employ `tree-sitter-java` to parse Java source files. To support the polymorphic nature of the Android framework, the parser explicitly extracts `extends` and `implements` clauses. These are resolved into `Extend` and `Implement` edges in the dependency graph, enabling the system to trace definitions up the inheritance chain.

Virtual Edge Resolution. To implement the resolution of implicit dependencies described in the methodology, the system introduces a deterministic heuristic layer during graph construction:

- **C Ambiguity Resolution**: When a header inclusion is ambiguous (e.g., multiple headers with the same name), the system calculates the directory distance between the source file and candidate targets. It creates an edge to the candidate with the minimum path distance, effectively identifying the "most likely" definition file as described in the reference approach.
- **Java Wildcard Resolution**: For wildcard imports (e.g., `import com.android.*`), the system scans the pre-built project index. It resolves the wildcard by mapping it to all concrete files within the corresponding logical package, converting implicit package knowledge into explicit `Import` edges.

E.2. Deterministic Context Retrieval

Consistent with the safety-critical requirement for exact signature matching, the retrieval mechanism replaces vector similarity with deterministic graph traversal.

- **Anchor Identification**: For every query, the system identifies the current file (Compilation Unit) and locates its corresponding node in the pre-built graph.
- **Depth-Limited Traversal (C)**: For C code, the system performs a recursive Depth-First Search (DFS) following `INCLUDE` and `TYPEOF` edges. The traversal depth is strictly controlled by a `MAX_HOP` parameter (defaulting to full traversal if unset). The retrieved contexts are ordered via pseudo-topological sort to maintain syntactic validity.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

- **Import and Inheritance Resolution (Java)**: For Java, the retrieval logic prioritizes explicit dependencies. It resolves specific imports to their physical file paths and navigates the inheritance edges constructed during the parsing phase to ensure inherited methods are available in the context.

E.3. Safety-Aligned Prompt Formation

To strictly constrain the LLM's generation space and prevent hallucination, we implement the Signature-Centric Abstraction strategy.

- **API Contract Extraction**: The system is configured with `ONLY_DEF = True`. Under this configuration, the retrieval engine strips away function bodies and implementation details.
- **Synthesized Views**:
  - For Java, the system generates an interface summary containing only Class headers, Method signatures, and Field declarations.
  - For C, it reconstructs a synthesized header view containing function prototypes and struct layouts.

This implementation ensures that the LLM receives the exact API contracts required for valid code generation while minimizing token consumption and strictly adhering to the safety protocols of the automotive domain.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.